生成を操る

temperature と top-k — 無難と暴走のあいだにツマミがある

rkskmt

前回の振り返り

- 訓練済みの重み（**`ckpt_best.pt`**）を手に入れ、文章が生成できるようになった
- モデルの出力は**確率分布** — 生成の自由度は「その分布から**どう引くか**」に集約される
- 今日は引き方のツマミを2つ握る。まずは準備：

Code

```
import torch
from gpt import GPT

torch.set_num_threads(1)
blob = torch.load("data/runs/neko/ckpt_best.pt", weights_only=True)
model = GPT(vocab_size=len(blob["vocab"]), **blob["cfg"])
model.load_state_dict(blob["state_dict"])
model.eval()
chars = blob["vocab"]
stoi = {c: i for i, c in enumerate(chars)}
```

- **gpt.py** と **ckpt_best.pt** は前回と同じ配置で

実験：毎回1位だけを選んだら

- 「一番ありそうな文字を選び続ける」のが最強に思える。バイグラムの Tweak で見た **argmax 作戦**

Code

```
def gen(prompt, n, temperature=1.0, top_k=None, seed=0):
    torch.manual_seed(seed)
    idx = torch.tensor([[stoi[c] for c in prompt]])
    out = model.generate(idx, n, temperature=temperature, top_k=top_k)
    return "".join(chars[i] for i in out[0].tolist())

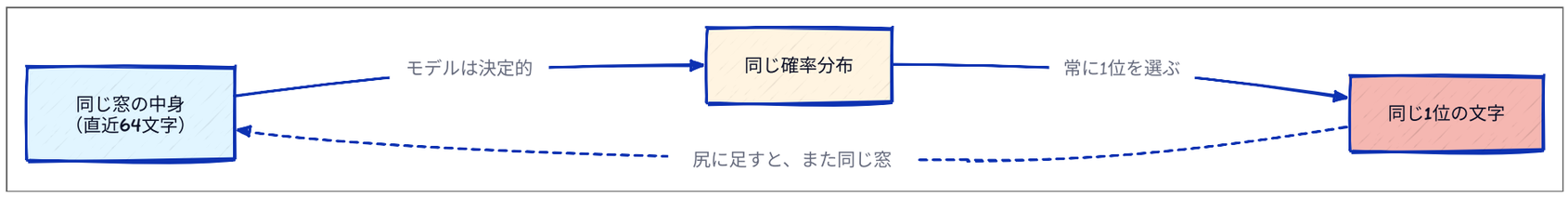
print(gen("吾輩は", 60, top_k=1))     # top_k=1 ＝ 常に1位だけ
```

Result

```
吾輩はこの猫のごとく、主人の頭をあらわしている。
「それでも構わんやい」と主人は少々ぷんぷんぷんぷんぷんしている。
「それでも
```

- 「少々**ぷんぷんぷんぷんぷん**している」 — **堂々巡りに落ちた**
- 常に最有力の道だけを歩くと、同じ角を回り続ける。しかもシードを変えても**毎回同じ文**

なぜ堂々巡りになるのか



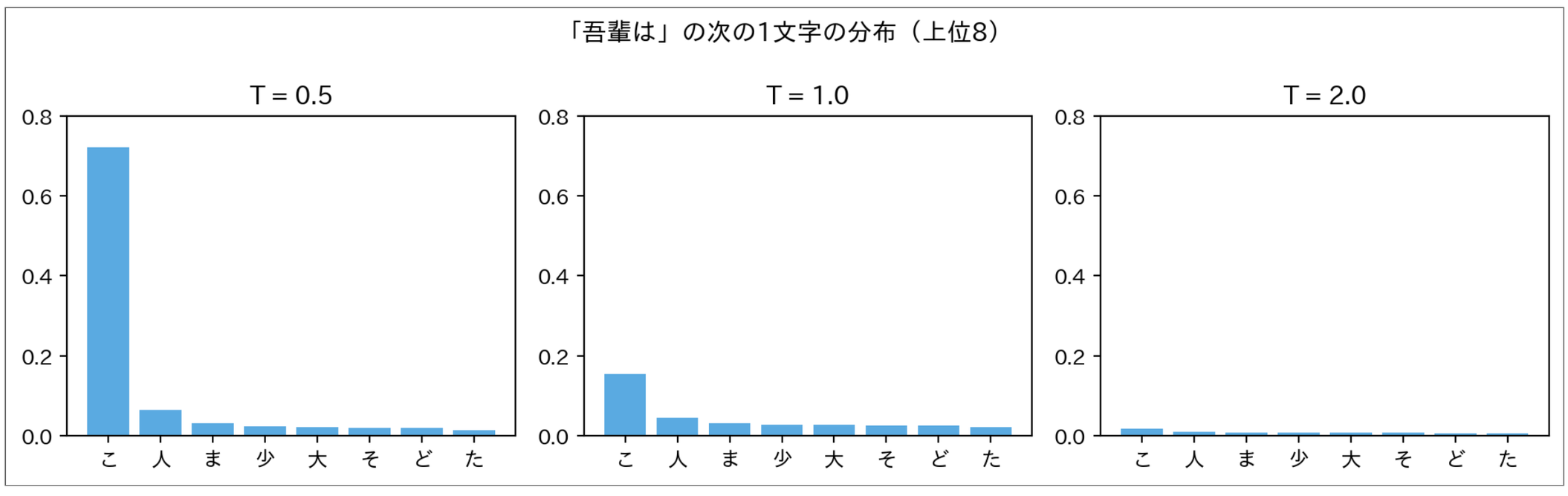
- モデルは同じ入力に**必ず同じ分布**を返す。1位だけ選ぶなら、次の文字も必ず同じ
- 生成中に「前に見た窓の中身」へ一度でも戻ると、**そこから先は永遠に同じ道** — 「ぷんぷん」の正体
- サイコロ（確率での抽選）は、このループを破る**唯一の乱数源**。問題は「どれくらい揺らすか」

ツマミ1：temperature

- softmax の**前**に、スコアを T で割る — たったこれだけ

$$\text{probs} = \text{softmax}\left(\frac{\text{logits}}{T}\right)$$

- $T < 1$：スコア差が拡大 → 分布が**尖る**（1位に集中、無難）
- $T > 1$：スコア差が縮小 → 分布が**平らになる**（番狂わせが増える、危険）
- $T \to 0$：ほぼ argmax。$T = 1$：モデルの分布そのまま



- 同じモデル・同じ入力でも、T だけで「こ」の確率が **72% ⇄ 16% ⇄ 2%** と変わる

T を手計算サイズで確かめる

- softmax の回のスコア (2, 1, −1) を、T で割ってから通すと：

Code

```python
import torch
import torch.nn.functional as F

s = torch.tensor([2.0, 1.0, -1.0])
for T in [0.5, 1.0, 2.0]:
    print(f"T={T}:", F.softmax(s / T, dim=0).round(decimals=3))
```

Result

```
T=0.5: tensor([0.8790, 0.1190, 0.0020])
T=1.0: tensor([0.7050, 0.2590, 0.0350])
T=2.0: tensor([0.5470, 0.3310, 0.1220])
```

- $T = 0.5$：スコア差が2倍に開いて1位が独走（0.88）。$T = 2.0$：差が半分に縮んで接戦（0.55 / 0.33 / 0.12）
- **モデルは1ミリも変わっていない**。変わったのは softmax に渡す前の「差の広げ方」だけ
- e^s は差の増幅器だった — T はその**増幅率のツマミ**

聞き比べ：どれがどの温度？

クイズ

T ＝ 0.2・0.8・2.0 のどれかで生成した3つ。どれがどの温度？

Result

```
--- A ---
吾輩は登ぱり何音野株か知るシ囃通り数が奇秘をも刑い台み掻…死絽を溜ごし嘔算吾私製霞如世一来出で』裁めて爆面
引る前将見田置し日武捐虞だ、飴ん悟り激纏珍騒振る所以早顔心で

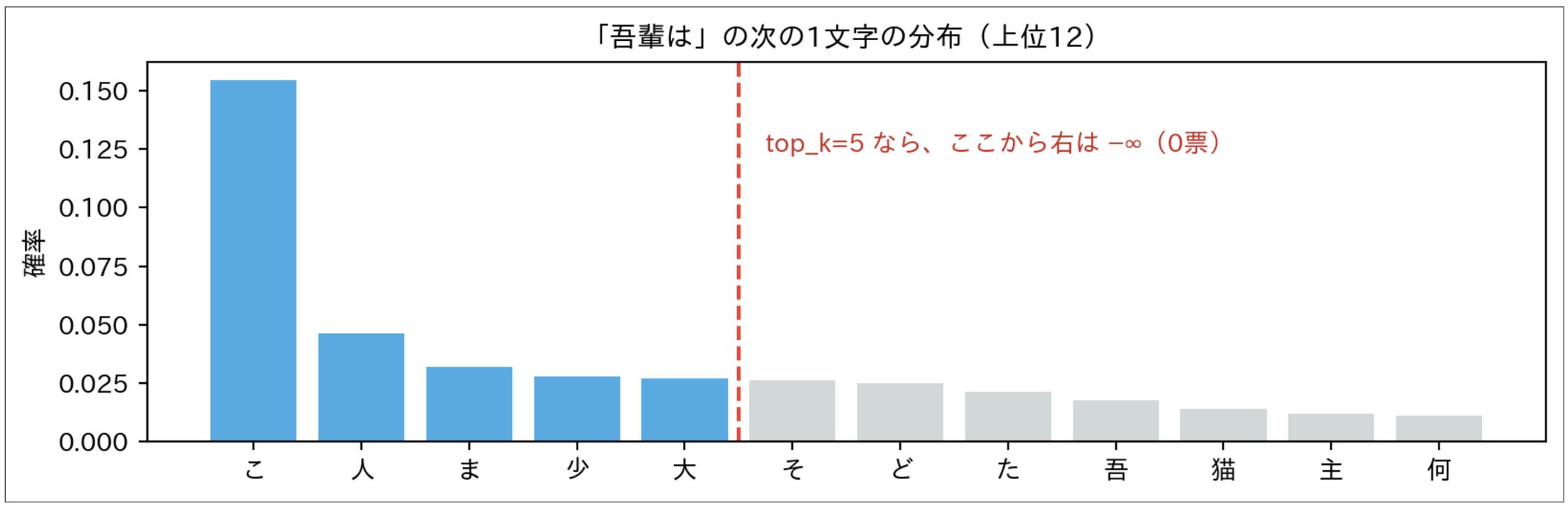
--- B ---
吾輩はこの猫を捉まっている。
　主人は「それでもいい、そんなになっても、あなたのです」と主人は少々焦れる。
「それではないか」と主人は「そんな事はないから、そんなに利

--- C ---
吾輩はあるまいと思うくらいならよりほかには元来主人のみを反対の穴があって、吾輩の頭をあつかない。それはこの
事なら、何しかしこまでも同じ事を悟ってはいっしゃるまい。また
```

- **A** は T=2.0：**漢字の豪雨** — 分布の尻尾にいたレア文字が当選しまくる
- **B** は T=0.2：文法は安定、しかし語彙が貧しく同じ言い回しを繰り返す（無難の刑）
- **C** は T=0.8：ほどよく揺れて、いちばん「漱石風」
- 作り方は3つとも **`gen("吾輩は", 80, temperature=T)`**（種は0で共通）

ツマミ2：top-k — 尻尾を切る安全網

- T を上げたいが暴走は困る → **上位 k 文字だけ残して**、あとは $-\infty$ にしてから抽選



- 票を $-\infty$ で消す — **因果マスクと同じ手口**（softmax の前に消すのが作法）

Code

```
print(gen("吾輩は", 80, temperature=2.0, top_k=40))
```

Result

```
吾輩はあるま品分に、かいるはずよりほといものも近い猫とし得るのを忘れしきりを捉まってあたかなかって見ましまか」
「ハハハハヴ」と念だ、細君もま時はよこはどっち構ってさ
```

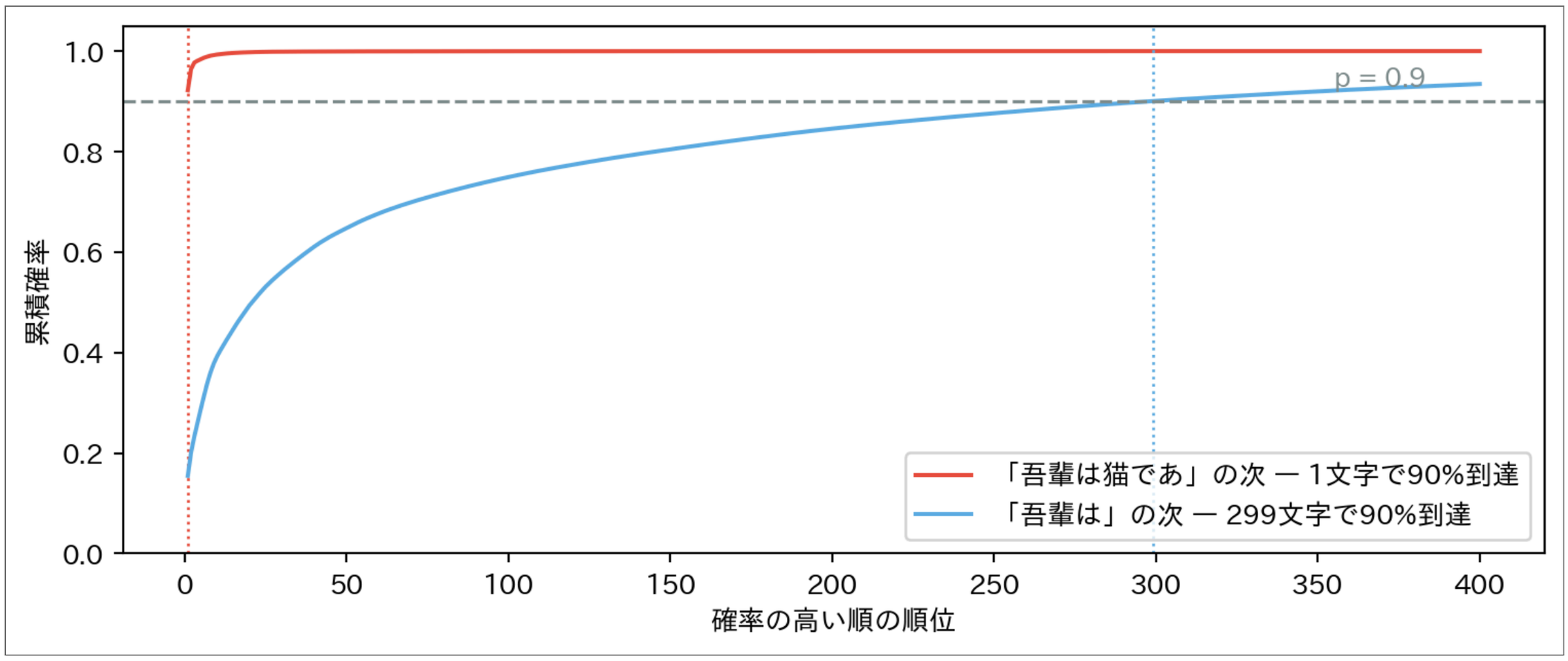
- 同じ T=2.0 でも、尻尾の3,005文字が消えるだけで「日本語の音」に戻ってくる
- **gpt.py** の **generate** はこの2ツマミ込みで書いてある：

Code

```
logits = logits / temperature                          # ツマミ1
if top_k is not None:
    v, _ = torch.topk(logits, top_k)
    logits[logits < v[:, [-1]]] = float("-inf")    # ツマミ2（尻尾を消す）
probs = F.softmax(logits, dim=-1)
nxt = torch.multinomial(probs, num_samples=1)      # サイコロ
```

top-p — 尻尾を「合計」で切る

- top-k の弱点：**常に k 席**の固定枠 — 自信満々の場面では広すぎ、大混戦では狭すぎる
- **top-p**（nucleus sampling）は席数でなく**得票の合計**で切る：確率の高い順に足していき、合計が p に達したら打ち止め



- 「吾輩は猫であ」の次は**1文字**（「る」92%）で到達、「吾輩は」の次は**299文字** — **枠の広さが分布の形に追随する**
- 自作GPTの **`generate`** のツマミは temperature と top-k の2つ。top-p は実物の API を読むための知識として持ち帰る

分布の形を、実物で歩く



- top-p のカーブが要約していたもの — **位置ごとの分布の形** — をエクスプローラで1つずつ覗ける



0. いま開いているのは「吾輩は」まで読んだ位置 — 低い棒がずらり（お手上げ型。1位「こ」でも 15.5%）
1. 「あ」（吾輩は猫であ）の位置をクリック — 「る」が **92%** でそびえる（確信型）
2. プリセットを会話文に替えて、いちばん平らな位置といちばん尖った位置を探す
3. 見つけた2箇所で「top-p ($p = 0.9$) の枠に何文字入るか」を目分量で予想 — 尖りは1桁、平らは3桁

エクスプローラを別タブで開く

サイコロの種 — 揺れを再現する

- 「確率的な生成」と「再現できる実験」は両立する — コンピュータの乱数は**疑似**乱数で、**種（シード）が同じなら出目の列も同じ**

Code

```
torch.manual_seed(0)
print(torch.multinomial(torch.tensor([0.7, 0.2, 0.1]), 8, replacement=True))
torch.manual_seed(0)
print(torch.multinomial(torch.tensor([0.7, 0.2, 0.1]), 8, replacement=True))
```

Result

```
tensor([2, 1, 0, 2, 0, 1, 0, 0])
tensor([2, 1, 0, 2, 0, 1, 0, 0])
```

Code

```
a = gen("吾輩は", 40, temperature=0.8, seed=7)
b = gen("吾輩は", 40, temperature=0.8, seed=7)   # 同じ種でもう一度
c = gen("吾輩は", 40, temperature=0.8, seed=8)   # 種を変える
print(a)
print("同じ種:", a == b, "／ 違う種:", a == c)
print(c)
```

Result

```
吾輩はようやくのは、融通の人のごときは違っている。主人は甚然として御互に願いて、まずし
同じ種: True ／ 違う種: False
吾輩は便宜者だから、ここを絞殺して見る小供はあって、これからでまた当人の二絃琴の真中は
```

- この講義のセルに毎回 **`manual_seed`** が入っていた種明かし — **スライドの出力を君の手元で完全再現するため**
- 「揺れ」がほしいときは種を変える。「証拠」にしたいときは種を固定して、**種の値も一緒に記録する**（レポートの作法）

実物も同じツマミで動いている

自作GPT	ChatGPT など実物の API
`temperature=0.8`	**`temperature`**（同じ意味・同じ数式）
`top_k=40`	**`top_k`** / **`top_p`**（前のスライドの2種類の尻尾切り）
`seed=7` で再現	**`seed`** 指定（対応するAPIもある）

- 「AIの回答は毎回違う」の正体は、この**サイコロ**
- 同じ質問に違う答えが返るのは故障ではなく、$T > 0$ の仕様

Tweak ツマミを回しまくる



- **gen** を好きに呼んで実験（実行のたびに **seed** を変えると別の文が出る）

1	**temperature=0.5 と 1.5 のあいだを刻む**	自分の耳で「ちょうどいい温度」を探す
2	**top_k=3 で temperature=1.0**	安全網がきつすぎると何の刑に近づく？
3	**gen("名前はまだ", 80, 0.8)**	書き出しで話題を誘導できるか試す

ここで観察

- **top_k** を小さくしすぎると argmax の「堂々巡り」に近づく — **2つのツマミは同じ「無難⇄暴走」軸**を別の方法で動かしている

まとめ

- 生成のツマミは2つ：**temperature**（分布を尖らせる/平らにする）と **top-k**（尻尾を切る）。実物にはもう1つ **top-p**（合計で切る可変枠）
- argmax（top_k=1）は堂々巡り、T が高すぎると漢字の豪雨 — **良い文はそのあいだ**にある
- 実装は「割る」「-∞で消す」「サイコロ」の3行。ChatGPT の同名パラメータも同じ仕組み
- 確率的な出力は故障ではなく仕様 — 種（シード）を固定すれば完全再現できる。今日の全セルがそうだった

現在の設計図　書き出し → 訓練済みGPT → **temperature・top-k・top-p** → 次の文字

次回予告

自作GPTはここで完成。次回は**このモデルと ChatGPT の距離**を測る — 足りないのは仕組みか、それとも量か。ついでに、LLM が堂々と嘘をつく理由も種明かしする。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：argmax の決定性

`top_k=1` で `seed=0` と `seed=123` の2回生成し、出力が**完全に一致する**ことを `==` で確かめよ。サイコロを振っているのに同じになる理由を1行で説明せよ。

問題2：安全網の値段

`temperature=1.0` で `top_k=5` と `top_k=None` の200文字を生成し、それぞれの**文字の種類数**（`len(set(...))`）を比べよ。top-k が「守ってくれるもの」と「奪うもの」を1行ずつ述べよ。

解答例

▶ 解答例を見る

Result

```
一致: True
top_k=5   : 67 種類
top_k=None: 110 種類
```